

EcoShop - Full Stack Web Development Capstone Project

Project Overview

EcoShop is a production-ready e-commerce web application built as the capstone project for the Full Stack Web Development course. It demonstrates mastery of modern web development concepts including responsive design, JavaScript DOM manipulation, RESTful patterns, persistent state management, and deployment best practices.

Key Features

Authentication & User Management

- **Simulated Authentication:** Demo login system with any username/password
- **Session Persistence:** User info stored in localStorage
- **User Profile Display:** Personalized welcome messages and dashboard

Product Management (CRUD)

- **Create Products:** Add new products with name, description, price, stock, and emoji icon
- **Read Products:** Display all products in responsive grid with stock status
- **Update Products:** Edit existing product details in-place
- **Delete Products:** Remove products with confirmation
- **Stock Management:** Track inventory with visual status indicators (In Stock / Low Stock / Out of Stock)

Shopping Cart

- **Add to Cart:** Add products with automatic cart badge updates
- **Quantity Control:** Increase/decrease quantities with +/- buttons
- **Remove Items:** Delete individual cart items
- **Real-Time Totals:** Automatic calculation of subtotal, tax (10%), and total
- **Cart Persistence:** Cart saved to localStorage across browser sessions

Order Management

- **Create Orders:** One-click checkout from cart

- **Order History:** View all placed orders with details
- **Order Cancellation:** Cancel orders and restore inventory
- **Order Status Tracking:** View order status (confirmed/cancelled)
- **Stock Deduction:** Automatic inventory adjustment on purchase

Responsive Design

- **Mobile-First Approach:** Fully functional on all devices
- **Fluid Grid System:** Adaptive layouts (1 → 2 → 3 → 4 columns at breakpoints)
- **Touch-Friendly UI:** Optimized buttons and controls for mobile
- **Zero Horizontal Scroll:** Perfect viewport management

Modern UI/UX

- **Design Tokens System:** Centralized color, typography, and spacing
- **Glassmorphism Effects:** Modern backdrop blur styling
- **Smooth Animations:** Fade-ins, slide-ups, transitions

- **Alert Notifications:** Success/error/info messages with auto-dismiss
- **Modal Dialogs:** Add/edit products in clean modal overlays
- **Empty States:** Helpful guidance when no data available

Persistent State Management

- **localStorage Integration:** All data persists across sessions
 - **Automatic Sync:** Changes automatically saved to browser storage
 - **Quota Management:** Error handling for storage limits
 - **Data Export:** Capability to export and import data
-

Project Structure

```
capstone-project/  
├─ capstone-index.html      # Main HTML  
file with all markup and styles  
├─ storage.js               # Storage  
manager (localStorage wrapper)  
├─ store.js                 # Business  
logic (CRUD, cart, orders)  
├─ capstone-app.js         # Application  
logic (UI controller)
```

```
|— CAPSTONE_README.md      # This file
|— ARCHITECTURE.md         # System
architecture documentation
|— DEPLOYMENT_GUIDE.md    # Deployment
instructions
|— API_DOCUMENTATION.md   # API
reference
|— TESTING_GUIDE.md       # Testing
checklist
|— .gitignore             # Git ignore
file
```

Quick Start

Local Development

1. Clone the Repository

```
git clone
https://github.com/yourusername/ecoshop
-capstone.git
cd ecoshop-capstone
```

2. Start Local Server

```
# Using Python 3
python -m http.server 8000
```

```
# OR using Node.js (if installed)
npx http-server
```

3. Open in Browser

- Navigate to
`http://localhost:8000/capstone-index.html`
- Login with any username and password

4. Explore Features

- Browse sample products
- Add/edit/delete products
- Add items to cart
- Checkout and create orders
- View order history

Demo Credentials

- **Username:** Any text (e.g., `john`, `alice`)
 - **Password:** Any text (e.g., `demo`, `password123`)
-

Architecture

Three-Tier Application Architecture

Presentation Layer (app.js)

- UI Rendering
- User Interactions
- View Management

↓

Business Logic Layer (store.js)

- CRUD Operations
- Cart Management
- Order Processing
- Validation & Calculations

↓

Data Layer (storage.js)

- localStorage Management

- Data Persistence
- Quota Handling

Module Responsibilities

storage.js (StorageManager)

- Encapsulates localStorage operations
- Provides type-safe get/set methods
- Handles JSON serialization
- Manages application data keys

store.js (Store)

- Core business logic
- Product CRUD operations
- Shopping cart management
- Order creation and processing
- Data calculations (totals, tax)
- Inventory management

app.js (Application)

- UI controller and state coordinator
- View switching and rendering

- Form handling
 - Event listeners
 - Alert/notification system
 - User authentication flow
-

Testing

Manual Testing Checklist

Authentication

- ☐ Login with any credentials
- ☐ Logout functionality
- ☐ User greeting displays correctly
- ☐ Session persists after page reload

Products

- ☐ View all products
- ☐ Add new product
- ☐ Edit existing product
- ☐ Delete product with confirmation
- ☐ Stock status displays correctly

Shopping Cart

- ☐ Add item to cart updates badge
- ☐ Remove item from cart

- ☐ Update quantity with +/- buttons
- ☐ Totals calculate correctly
- ☐ Cart persists after page reload

Orders

- ☐ Create order from cart
- ☐ Order appears in order history
- ☐ Order details display correctly
- ☐ Cancel order restores inventory
- ☐ Stock updates after purchase

Responsive Design

- ☐ Test on mobile (320px)
- ☐ Test on tablet (768px)
- ☐ Test on desktop (1024px+)
- ☐ No horizontal scrollbars
- ☐ Touch controls work on mobile

Error Handling

- ☐ Add to cart with insufficient stock
- ☐ Invalid form inputs
- ☐ Add product with missing fields
- ☐ Remove non-existent items



Deployment

Deploy to Vercel (Recommended)

1. Push to GitHub

```
git push origin main
```

2. Connect to Vercel

- Go to vercel.com
- Click "Import Project"
- Connect your GitHub repository
- Select project folder (if in subdirectory)
- Deploy!

3. Access Live URL

- Vercel provides automatic URL
- Example: `https://ecoshop-capstone.vercel.app`

Deploy to Netlify

1. Build for Production (optional - not needed for static files)

```
# Just push static files to Netlify
```

2. Drag & Drop Deploy

- Go to netlify.com
- Click "Create a new site"

- Drag your project folder onto the drop zone
- Deploy automatically!

3. **Connect GitHub** (optional)

- Site settings → Build & Deploy
- Connect GitHub repository
- Auto-deploy on push

Deploy to Cloudflare Pages

1. **Install Wrangler** (Cloudflare CLI)

```
npm install -g wrangler
```

2. **Deploy**

```
wrangler pages deploy .
```

3. **Access Live URL**

- Cloudflare provides automatic URL
- Example: `https://ecoshop-capstone.pages.dev`

Security Considerations

Implemented Security Measures

- **XSS Prevention:** HTML escaping for all dynamic content
- **Input Validation:** Form validation and sanitization
- **CSRF Protection:** No external API calls (localStorage only)
- **localStorage Limits:** Graceful handling of quota exceeded errors
- **No Sensitive Data:** Demo-only authentication, no real credentials

Production Recommendations

- Implement server-side authentication
- Use HTTPS for all connections
- Add CORS headers properly
- Implement rate limiting
- Add server-side input validation
- Use Content Security Policy headers
- Regular security audits

Browser Compatibility

Browser	Supported	Notes
Chrome	 Yes	Latest 2 versions

Browser	Supported	Notes
Firefox	✓ Yes	Latest 2 versions
Safari	✓ Yes	Latest 2 versions
Edge	✓ Yes	Latest 2 versions
IE 11	✗ No	Requires transpilation

Required Features:

- ES6 (Arrow functions, template literals, classes)
- localStorage API
- Fetch API (or XMLHttpRequest)
- CSS Grid/Flexbox



Performance Metrics

Optimization Techniques

- **No External Dependencies:** Pure vanilla JavaScript
- **Minimal CSS:** All CSS in single file (~100KB)
- **Efficient DOM:** Class-based rendering with innerHTML
- **Local-First Data:** All data in localStorage (instant access)

- **Lazy Loading:** Products rendered on-demand

Performance Targets

- **First Contentful Paint:** < 1s
 - **Interactive:** < 1.5s
 - **Total Bundle Size:** ~60KB (HTML + CSS + JS combined)
 - **Lighthouse Score:** 90+
-

Development Workflow

Adding a New Feature

1. **Update HTML** (if needed)
 - Add new elements with IDs and classes
 - Follow existing naming conventions
2. **Update store.js** (if needed)
 - Add business logic methods
 - Update data structure if needed
3. **Update app.js**
 - Add event listeners
 - Add render methods
 - Handle user interactions

4. Test Locally

- Run local server
- Test all browsers
- Test responsiveness

5. Commit and Push

```
git add .  
git commit -m "feat: Add new feature"  
git push origin main
```

Modifying Styles

1. **Edit Capstone-index.html** CSS section
 2. **Follow Design Tokens**
 - Use CSS custom properties
 - Maintain consistency
 3. **Test on all breakpoints**
 4. **Commit changes**
-

Learning Outcomes

By building this capstone project, you have demonstrated:

1. **HTML/CSS Skills**

- Semantic markup
- CSS Grid and Flexbox
- Responsive design
- Design systems
- Accessibility basics

2. JavaScript Skills

- ES6+ features (classes, arrow functions, template literals)
- DOM manipulation
- Event handling
- Async operations
- Error handling

3. Web Development Concepts

- MVC architecture pattern
- Persistent state management
- Local-first data strategies
- CRUD operations
- Form validation

4. Professional Skills

- Git version control
- Project documentation
- Testing methodology
- Deployment procedures

- Code organization

Contributing

1. Fork the repository
2. Create feature branch (`git checkout -b feature/amazing-feature`)
3. Commit changes (`git commit -m 'Add amazing feature'`)
4. Push to branch (`git push origin feature/amazing-feature`)
5. Open Pull Request

License

This project is open source and available under the MIT License.

Course Information

Course: Full Stack Web Development

Institution: RabTech Academy

Level: Capstone Project (Final)

Duration: 8+ weeks

Support

For questions or issues:

1. Check the [ARCHITECTURE.md](#) for system design details
 2. Review [API_DOCUMENTATION.md](#) for module reference
 3. Consult [DEPLOYMENT_GUIDE.md](#) for deployment help
 4. See [TESTING_GUIDE.md](#) for testing procedures
-

Submission Checklist

- ☒ All features implemented
 - ☒ Responsive design completed
 - ☒ localStorage integration working
 - ☒ CRUD operations functional
 - ☒ Error handling in place
 - ☒ Documentation complete
 - ☒ Code organized and clean
 - ☒ Deployed to live URL
 - ☐ GitHub repository submitted
 - ☐ Live deployment URL submitted
-

**Built with ❤️ as a Full Stack Web Development
Capstone**

Last Updated: September 24, 2026